

I am doing a VibeCoding lab assignment for this repository:

 [SETAPESU26/10_breakout](#)

I have cloned my own fork and I am working on it locally in VS Code.

I have already recorded the required BEFORE-change video.

Please inspect the repository and README carefully before making any changes.

Now implement ONLY Task 1:

Task 1: Fix ball–brick collision

Current behavior:

- The ball bounces off bricks normally.
- A brick has hits_remaining.
- When the ball hits a brick, hits_remaining is decreased.
- However, the brick is never removed from the board.

Required behavior:

- When the ball hits a brick, decrease hits_remaining.
- When hits_remaining reaches 0, remove that brick from the active board.
- The "Bricks left" counter must decrease correctly.

Important:

- Preserve the existing architecture.
- Make minimal changes.
- Do not implement Task 2, Task 3, or Task 4.
- Do not add lives, Game Over, restart, brick types, score, or combo functionality.
- Do not unnecessarily modify unrelated files.

After implementing Task 1:

1. Tell me which files you modified.

2. Briefly explain the changes.
3. Check for syntax/runtime errors.
4. Give me a short manual testing checklist for Task 1.

Stop after Task 1. Do not implement any other task.

Show less ^



Worked for 19s >

I inspected the repository and README carefully. The repository explicitly says Task 1's bug is in `breakout/game/game_engine.py`: `hits_remaining` is decremented, but the brick is never removed.  GitHub +1

I cannot directly edit the files inside your local VS Code workspace from this chat, so I won't pretend the local change was made. The exact minimal Task 1 change you should make is:

1. Modify only `breakout/game/game_engine.py`

Find:

Python

Run

```
for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        brick.hits_remaining -= 1 # BUG: tracked, but never actually checked to remove the
        break
```

Replace it with:

Python

Run

```
for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        brick.hits_remaining -= 1
        if brick.hits_remaining <= 0:
            self.bricks.remove(brick)
        break
```

That's the **only code change required for Task 1**.

The existing `draw()` already calculates the counter from:

Python

Run

```
len(self.bricks)
```

so once the destroyed brick is removed, "Bricks left" automatically decreases correctly.



Files modified

Only:

```
breakout/game/game_engine.py
```

Do **not** modify:

- brick.py
- collision.py
- ball.py
- renderer.py
- main.py

The collision function already returns `True` when the ball hits a brick and reverses the ball's vertical velocity, so it does not need modification.  GitHub

Syntax check

After making the change in VS Code, open the terminal at the `breakout` directory and run:

Bash

```
python -m py_compile game/game_engine.py
```

If there is **no output**, the file passed Python's syntax check.

Then run:

Bash

```
python main.py
```

The README specifies Python 3.10+ and `pygame>=2.5.0`.  GitHub +1

Manual testing checklist — Task 1 only

1. Start the game.
2. Confirm the **Bricks left** counter initially shows the full number of bricks.
3. Let the ball hit a brick.
4. Confirm the brick disappears when its `hits_remaining` reaches `0`.
5. Confirm **Bricks left decreases by 1**.

6. Confirm the ball still bounces normally after hitting the brick.
7. Hit another brick and verify the same behavior.
8. Make sure **no lives, Game Over, restart, score, combo, or new brick types** appear.

For this starter repository, bricks currently default to `hits_remaining=1`, so each successful brick collision should remove that brick immediately.  [GitHub](#)

Stop here for Task 1.



Task 1 is now completed and manually tested successfully.

I verified that:

- The ball collides with bricks.
- A brick is removed when its `hits_remaining` reaches 0.
- The "Bricks left" counter decreases correctly.
- Task 1 works as required.

Now implement **ONLY** Task 2 and Task 3 from the README.

Do NOT implement Task 4 yet.

TASK 2 — Implement lives and game over

Required:

- Add a 3-life system.
- Player starts with 3 lives.
- When the ball falls below the paddle, decrease lives by 1.
- Reset the ball for another attempt if lives remain.
- Display the remaining lives.
- When all 3 lives are used, end the game with a clear Game Over state.
- Provide a way to restart the game after Game Over, preferably using R.

TASK 3 — Add different brick types

Required:

- Normal brick: destroyed after one hit.
- Strong brick: requires multiple hits before it is destroyed.

- Unbreakable brick: never destroyed, no matter how many times it is hit.
- Give each brick type a visually different appearance.
- Make sure each type behaves correctly when hit.

Important:

- Task 1 is already working. Preserve its behavior.
- Preserve the existing project architecture.
- Make minimal changes to the existing code.
- Do not rewrite the game unnecessarily.
- Do not implement Task 4.
- Do NOT add score or combo functionality yet.
- Do NOT add unrelated features.
- Reuse the existing collision, rendering, ball, paddle, and game-engine structure wherever possible.
- Make sure Task 2 and Task 3 work together with the already completed Task 1.

Before making changes, briefly identify which existing files need to be modified and why.

After implementation:

1. Tell me exactly which files were modified.
2. Briefly explain the changes for Task 2 and Task 3.
3. Check for Python syntax/runtime errors.
4. Give me a concise manual testing checklist covering:
 - 3 lives
 - losing a life
 - ball reset
 - Game Over
 - R restart
 - Normal brick
 - Strong brick
 - Unbreakable brick
 - visually different brick types
 - Bricks left counter
5. Confirm that Task 4 (score/combo) was NOT implemented.

Stop after Task 2 and Task 3.

Show less ^

Worked for 10s >

I inspected the README and the current repository structure before making the changes. The README confirms that Task 2 is lives/game-over/restart and Task 3 is the three brick types; Task 4 is specifically score/combo and will remain untouched.  GitHub

Because your actual fork is a **local VS Code workspace**, I can inspect the public repository but I cannot directly write into your local filesystem from this chat. So I can't truthfully claim that I modified your local files. Below is the **exact minimal implementation** to apply.

Files that need modification

File	Why
breakout/game/game_engine.py	Lives, ball reset, Game Over/restart, brick creation/type behavior
breakout/game/brick.py	Add brick type information and type-specific hit behavior
breakout/game/renderer.py	Display lives and Game Over banner

main.py does **not** need modification because it already forwards keyboard events to engine.handle_keydown() and calls engine.update() / draw() .  GitHub

collision.py also does not need modification because it already detects a collision and handles the bounce.  GitHub

1. breakout/game/brick.py

Replace the file with:

Python	Run
<pre>""" Brick: a single block with different durability types. """ import pygame class Brick: def __init__(self, x, y, width,</pre>	

```

        height,
        brick_type="normal",
        hits_remaining=1,
        color=(200, 90, 90),
    ):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.brick_type = brick_type
        self.hits_remaining = hits_remaining
        self.color = color

    def get_rect(self):
        return pygame.Rect(int(self.x), int(self.y), self.width, self.height)

    def hit(self):
        """Apply one hit and return whether the brick should be destroyed."""
        if self.brick_type == "unbreakable":
            return False

        self.hits_remaining -= 1
        return self.hits_remaining <= 0

```

This keeps the existing `hits_remaining` architecture while giving each brick a type.

2. breakout/game/game_engine.py

Make these changes.

Add lives in `__init__`

Change:

Python

Run

```

def __init__(self):
    self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)
    self.bricks = self._build_bricks()

```

to:

Python

Run

```

def __init__(self):
    self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)
    self.bricks = self._build_bricks()

```

```
self.lives = 3
self.game_over = False
```

Replace _build_bricks()

Use:

Python

Run

```
def _build_bricks(self):
    bricks = []
    total_width = BRICK_COLS * (BRICK_WIDTH + BRICK_GAP) - BRICK_GAP
    start_x = (WIDTH - total_width) / 2

    for row in range(BRICK_ROWS):
        for col in range(BRICK_COLS):
            x = start_x + col * (BRICK_WIDTH + BRICK_GAP)
            y = BRICK_TOP_MARGIN + row * (BRICK_HEIGHT + BRICK_GAP)

            if row == 0:
                brick_type = "unbreakable"
                hits_remaining = 999999
                color = (100, 100, 100)
            elif row == 1:
                brick_type = "strong"
                hits_remaining = 3
                color = (230, 160, 60)
            else:
                brick_type = "normal"
                hits_remaining = 1
                color = (200, 90, 90)

            bricks.append(
                Brick(
                    x,
                    y,
                    BRICK_WIDTH,
                    BRICK_HEIGHT,
                    brick_type=brick_type,
                    hits_remaining=hits_remaining,
                    color=color,
                )
            )

    return bricks
```

So the rows become:

- Row 1 → **Unbreakable**
- Row 2 → **Strong**, 3 hits
- Rows 3–4 → **Normal**, 1 hit

This gives each type a clearly different appearance.

Keep `_reset_ball()`

Python

Run

```
def _reset_ball(self):  
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)
```

Implement R restart

Replace:

Python

Run

```
def handle_keydown(self, key):  
    pass
```

with:

Python

Run

```
def handle_keydown(self, key):  
    if key == pygame.K_r and self.game_over:  
        self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)  
        self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)  
        self.bricks = self._build_bricks()  
        self.lives = 3  
        self.game_over = False
```

Update brick collision

Replace your Task 1 collision section:

Python

Run

```
for brick in self.bricks:  
    if handle_ball_brick_collision(self.ball, brick):  
        brick.hits_remaining -= 1  
        if brick.hits_remaining <= 0:  
            self.bricks.remove(brick)  
        break
```

with:

Python

Run

```

for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        if brick.hit():
            self.bricks.remove(brick)
        break

```

This preserves Task 1 while adding the different brick behaviors.

Replace update()

Use:

Python

Run

```

def update(self):
    if self.game_over:
        return

    self.ball.update()
    self.ball.bounce_off_walls(WIDTH)

    if self.ball.get_rect().colliderect(self.paddle.get_rect()) and self.ball.vy > 0:
        self.ball.bounce_off_paddle(self.paddle.get_rect())

    for brick in self.bricks:
        if handle_ball_brick_collision(self.ball, brick):
            if brick.hit():
                self.bricks.remove(brick)
            break

    if self.ball.is_below(HEIGHT):
        self.lives -= 1

        if self.lives > 0:
            self._reset_ball()
        else:
            self.game_over = True

```

Notice that **Task 4 is not involved** here. There is no score or combo state.

Replace draw()

Use:

Python

Run

```

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,

```

```

        self.paddle,
        self.ball,
        self.bricks,
    )

    renderer.draw_text(
        surface,
        font,
        f"Bricks left: {len(self.bricks)}",
        (10, 10),
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 35),
    )

    if self.game_over:
        renderer.draw_banner(
            surface,
            font,
            "GAME OVER - Press R to Restart",
        )

```

The existing renderer already has `draw_banner()` , so we can reuse it rather than creating a new rendering system.  [GitHub](#)

3. breakout/game/renderer.py

No modification is actually required.

The existing renderer already:

- draws `brick.color`
- draws the paddle
- draws the ball
- has `draw_text()`
- has `draw_banner()`

So the Game Engine can reuse all of it.  [GitHub](#)

That means the final modified-file list is actually only:

```

breakout/game/game_engine.py
breakout/game/brick.py

```

4. Syntax check

From your VS Code terminal, inside the `breakout` directory:

Bash

```
python -m py_compile game/game_engine.py game/brick.py
```

If it produces **no output**, the syntax check passed.

Then run:

Bash

```
python main.py
```

The README specifies Python 3.10+ and running the game with `python main.py`.



Manual testing checklist

Task 2 — Lives

- ☐ Game starts with **Lives: 3**
- ☐ Let the ball fall below the paddle
- ☐ Lives changes **3 → 2**
- ☐ Ball returns to its starting position
- ☐ Lose the ball again → **2 → 1**
- ☐ Lose the ball again → **1 → 0**
- ☐ **GAME OVER** appears
- ☐ Ball/game movement stops after Game Over
- ☐ Press **R**
- ☐ Game restarts with **Lives: 3**
- ☐ Bricks are rebuilt after restart

Task 3 — Brick types

- ☐ **Normal** bricks are visually different
- ☐ Normal brick disappears after **1 hit**
- ☐ **Strong** bricks have a different appearance
- ☐ Strong brick survives the first hit
- ☐ Strong brick survives the second hit

- ☐ Strong brick disappears after the third hit
- ☐ **Unbreakable** bricks have a different appearance
- ☐ Unbreakable bricks remain after repeated hits
- ☐ Unbreakable bricks do not reduce the `Bricks left` counter
- ☐ Destroying a normal/strong brick decreases `Bricks left`
- ☐ Task 1 behavior remains intact

Explicitly NOT implemented

Task 4 — `score/combo` has NOT been implemented.

There is no score, combo multiplier, combo tracking, or score display in these changes.

The README defines those as the separate Task 4.  [GitHub](#)

     ...  Sources

I have completed and manually tested Tasks 1, 2, and 3.

Current completed functionality:

- Task 1: Bricks are removed when `hits_remaining` reaches 0, and the "Bricks left" counter decreases correctly.
- Task 2: 3 lives, life decrement when the ball is missed, ball reset, Game Over, and R restart.
- Task 3: Normal, Strong, and Unbreakable brick types with different appearances and behaviors.

Now implement ONLY Task 4 from the README and then perform a final review of all four tasks.

TASK 4 — Implement combo scoring

Required behavior from the README:

- Add a score.
- Add a combo multiplier.
- Destroying bricks back-to-back without missing the ball in between should increase the combo multiplier.
- Each subsequent brick destruction should be worth more points.
- Missing the ball should reset the combo.
- Display both the score and the current multiplier.

Important:

- Preserve all existing working functionality from Tasks 1, 2, and 3.
- Make minimal changes to the existing architecture.
- Do NOT rewrite the game unnecessarily.
- Do NOT change the existing collision architecture unless absolutely necessary.
- Do NOT remove or alter the 3-life/Game Over/restart system.
- Do NOT change the Normal, Strong, or Unbreakable brick behavior.
- Do NOT add unrelated features.
- Do not add extra gameplay mechanics that are not required by the README.

For scoring:

- A brick should award points only when it is actually destroyed, not merely when it is hit.
- A Strong brick should therefore award points only when its final required hit destroys it.
- An Unbreakable brick should never award destruction points because it is never destroyed.
- Destroying consecutive bricks without losing the ball should increase the combo multiplier.
- When the ball falls below the paddle and a life is lost, reset the combo/multiplier.
- Choose a simple, reasonable scoring scheme consistent with the existing code. Do not over-engineer it.

Display:

- Keep the existing "Bricks left" display.
- Keep the existing Lives display.
- Add the current Score.
- Add the current Combo multiplier.
- Keep the Game Over message and R restart behavior.

After implementing Task 4:

1. Tell me exactly which files were modified.
2. Briefly explain the Task 4 changes.
3. Check for Python syntax/runtime errors.
4. Confirm that Tasks 1, 2, 3, and 4 are all still present.
5. Specifically verify:
 - Normal brick destruction gives score.

- Strong brick gives score only when destroyed.
- Unbreakable brick gives no destruction score.
- Consecutive brick destruction increases the multiplier.
- Missing the ball resets the multiplier.
- Score and multiplier are displayed.
- Lives and Game Over still work.
- R restart still works.
- Bricks left still decreases correctly.

6. Confirm that no unrelated features were added.

Do not make any changes beyond Task 4 and the minimum fixes required to preserve the already completed Tasks 1–3.

Show less ^

Yes. Since Tasks 1–3 are already completed in your local VS Code project, **Task 4 should only add score/combo logic on top of the existing implementation.**

I can inspect the public repository, but I cannot directly modify your local fork/filesystem from this chat. So I won't claim that I changed your local files. Apply the following minimal changes to your existing `game_engine.py`.

File to modify

Only:

`breakout/game/game_engine.py`

You should **not** need to modify `brick.py`, `collision.py`, `renderer.py`, or `main.py`.

1. Add score and combo state

Inside `__init__`, alongside your existing lives/game-over state:

Python

Run

```
self.lives = 3
self.game_over = False

self.score = 0
self.combo = 1
```

2. Reset score/combo appropriately

For a normal life loss, **only reset the combo**, not the score.

In the existing ball-missed section:

Python

Run

```
if self.ball.is_below(HEIGHT):
    self.lives -= 1

    if self.lives > 0:
        self._reset_ball()
    else:
        self.game_over = True
```

change it to:

Python

Run

```
if self.ball.is_below(HEIGHT):
    self.lives -= 1
    self.combo = 1

    if self.lives > 0:
        self._reset_ball()
    else:
        self.game_over = True
```

This satisfies the requirement that missing the ball resets the multiplier while preserving the score.

3. Award points only when a brick is destroyed

Your current Task 3 collision logic should look approximately like:

Python

Run

```
for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        if brick.hit():
            self.bricks.remove(brick)
        break
```

Change **only this section** to:

Python

Run

```
for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        if brick.hit():
```

```

self.bricks.remove(brick)

self.score += 100 * self.combo
self.combo += 1

break

```

This is important:

- Normal brick → destroyed → $+100 \times \text{combo}$
- Strong brick → hit but survives → **0 points**
- Strong brick → final hit destroys it → $+100 \times \text{combo}$
- Unbreakable brick → `brick.hit()` returns `False` → **0 points**
- Next destroyed brick without losing the ball → higher multiplier

For example:

```

1st destroyed brick → 100 points    (x1)
2nd destroyed brick → 200 points    (x2)
3rd destroyed brick → 300 points    (x3)
4th destroyed brick → 400 points    (x4)

```

The multiplier therefore represents the value of the **next destroyed brick**.

4. Display Score and Combo

Keep your existing displays:

Python

Run

```

renderer.draw_text(
    surface,
    font,
    f"Bricks left: {len(self.bricks)}",
    (10, 10),
)

renderer.draw_text(
    surface,
    font,
    f"Lives: {self.lives}",
    (10, 35),
)

```

Add:

Python

Run

```

renderer.draw_text(
    surface,
    font,
    f"Score: {self.score}",
    (10, 60),
)

renderer.draw_text(
    surface,
    font,
    f"Combo: x{self.combo}",
    (10, 85),
)

```

Keep your existing Game Over code unchanged:

Python

Run

```

if self.game_over:
    renderer.draw_banner(
        surface,
        font,
        "GAME OVER - Press R to Restart",
    )

```

5. Make sure R restart resets score/combo

Your existing restart code probably looks like:

Python

Run

```

if key == pygame.K_r and self.game_over:
    self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)
    self.bricks = self._build_bricks()
    self.lives = 3
    self.game_over = False

```

Add:

Python

Run

```

self.score = 0
self.combo = 1

```

so it becomes:

Python

Run

```

if key == pygame.K_r and self.game_over:
    self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)
    self.bricks = self._build_bricks()
    self.lives = 3
    self.score = 0
    self.combo = 1
    self.game_over = False

```

That keeps restart behavior consistent with starting a fresh game.

Final code logic

Your important collision/life section should now effectively be:

Python

Run

```

for brick in self.bricks:
    if handle_ball_brick_collision(self.ball, brick):
        if brick.hit():
            self.bricks.remove(brick)

            self.score += 100 * self.combo
            self.combo += 1

        break

if self.ball.is_below(HEIGHT):
    self.lives -= 1
    self.combo = 1

    if self.lives > 0:
        self._reset_ball()
    else:
        self.game_over = True

```

This preserves the existing Task 1 + Task 3 architecture rather than moving collision logic elsewhere.

Syntax check

From the `breakout` directory:

Bash

```
python -m py_compile game/game_engine.py game/brick.py game/collision.py game/renderer.py
```

Then run:

Bash

```
python main.py
```

If `py_compile` produces no output, there are no Python syntax errors.

Final review checklist

Requirement	Expected result
Task 1	Destroyed bricks disappear
Bricks left	Decreases only when a brick is destroyed
Task 2	Starts with 3 lives
Life loss	Ball resets; lives decrease
Combo reset	Combo returns to $\times 1$ after missing
Game Over	Appears after all lives are lost
R restart	Restores lives, bricks, score and combo
Task 3 Normal	One hit $\rightarrow$ destroyed $\rightarrow$ score
Task 3 Strong	Intermediate hits $\rightarrow$ no score
Strong final hit	Destroyed $\rightarrow$ score awarded
Unbreakable	Never destroyed $\rightarrow$ no score
Combo	Consecutive destroyed bricks increase multiplier
Score	Displayed
Multiplier	Displayed
Lives	Still displayed
Game Over	Still displayed
Task 4 only	No unrelated mechanics

Files modified

breakout/game/game_engine.py

Tasks present after the change

- Task 1: ☒ Brick destruction
- Task 2: ☒ Lives + Game Over + R restart
- Task 3: ☒ Normal + Strong + Unbreakable bricks
- Task 4: ☒ Score + combo multiplier

No score/combo changes require altering the existing collision architecture, and **no unrelated features are added.**



ChatGPT can make mistakes. Check important info